

designed with asynchronous operations in mind, allowing you to easily define steps that can run in parallel.

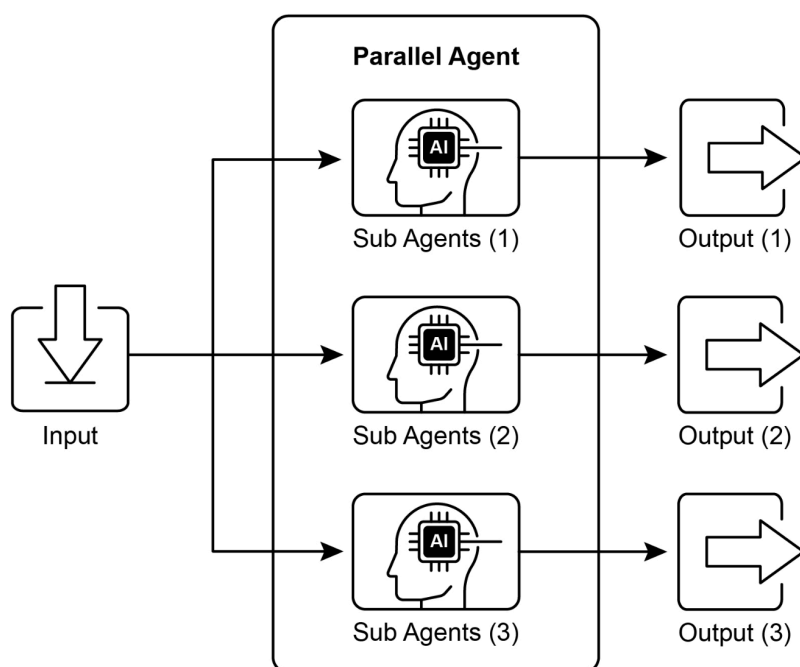


Fig.1. Example of parallelization with sub-agents

Frameworks like LangChain, LangGraph, and Google ADK provide mechanisms for parallel execution. In LangChain Expression Language (LCEL), you can achieve parallel execution by combining runnable objects using operators like `|` (for sequential) and by structuring your chains or graphs to have branches that execute concurrently. LangGraph, with its graph structure, allows you to define multiple nodes that can be executed from a single state transition, effectively enabling parallel branches in the workflow. Google ADK provides robust, native mechanisms to facilitate and manage the parallel execution of agents, significantly enhancing the efficiency and scalability of complex, multi-agent systems. This inherent capability within the ADK framework allows developers to design and implement solutions where multiple agents can operate concurrently, rather than sequentially.

The Parallelization pattern is vital for improving the efficiency and responsiveness of agentic systems, especially when dealing with tasks that involve multiple independent lookups, computations, or interactions with external services. It's a key technique for optimizing the performance of complex agent workflows.

Practical Applications & Use Cases

Parallelization is a powerful pattern for optimizing agent performance across various applications:

1. Information Gathering and Research:

Collecting information from multiple sources simultaneously is a classic use case.

- **Use Case:** An agent researching a company.
 - **Parallel Tasks:** Search news articles, pull stock data, check social media mentions, and query a company database, all at the same time.
 - **Benefit:** Gathers a comprehensive view much faster than sequential lookups.

2. Data Processing and Analysis:

Applying different analysis techniques or processing different data segments concurrently.

- **Use Case:** An agent analyzing customer feedback.
 - **Parallel Tasks:** Run sentiment analysis, extract keywords, categorize feedback, and identify urgent issues simultaneously across a batch of feedback entries.
 - **Benefit:** Provides a multi-faceted analysis quickly.

3. Multi-API or Tool Interaction:

Calling multiple independent APIs or tools to gather different types of information or perform different actions.

- **Use Case:** A travel planning agent.
 - **Parallel Tasks:** Check flight prices, search for hotel availability, look up local events, and find restaurant recommendations concurrently.
 - **Benefit:** Presents a complete travel plan faster.

4. Content Generation with Multiple Components:

Generating different parts of a complex piece of content in parallel.

- **Use Case:** An agent creating a marketing email.
 - **Parallel Tasks:** Generate a subject line, draft the email body, find a relevant image, and create a call-to-action button text simultaneously.
 - **Benefit:** Assembles the final email more efficiently.

5. Validation and Verification:

Performing multiple independent checks or validations concurrently.

- **Use Case:** An agent verifying user input.
 - **Parallel Tasks:** Check email format, validate phone number, verify address against a database, and check for profanity simultaneously.
 - **Benefit:** Provides faster feedback on input validity.

6. Multi-Modal Processing:

Processing different modalities (text, image, audio) of the same input concurrently.

- **Use Case:** An agent analyzing a social media post with text and an image.
 - **Parallel Tasks:** Analyze the text for sentiment and keywords *and* analyze the image for objects and scene description simultaneously.
 - **Benefit:** Integrates insights from different modalities more quickly.

7. A/B Testing or Multiple Options Generation:

Generating multiple variations of a response or output in parallel to select the best one.

- **Use Case:** An agent generating different creative text options.
 - **Parallel Tasks:** Generate three different headlines for an article simultaneously using slightly different prompts or models.
 - **Benefit:** Allows for quick comparison and selection of the best option.

Parallelization is a fundamental optimization technique in agentic design, allowing developers to build more performant and responsive applications by leveraging concurrent execution for independent tasks.

Hands-On Code Example (LangChain)

Parallel execution within the LangChain framework is facilitated by the LangChain Expression Language (LCEL). The primary method involves structuring multiple runnable components within a dictionary or list construct. When this collection is passed as input to a subsequent component in the chain, the LCEL runtime executes the contained runnables concurrently.

In the context of LangGraph, this principle is applied to the graph's topology. Parallel workflows are defined by architecting the graph such that multiple nodes, lacking direct sequential dependencies, can be initiated from a single common node. These parallel pathways execute independently before their results can be aggregated at a subsequent convergence point in the graph.

The following implementation demonstrates a parallel processing workflow constructed with the LangChain framework. This workflow is designed to execute two

independent operations concurrently in response to a single user query. These parallel processes are instantiated as distinct chains or functions, and their respective outputs are subsequently aggregated into a unified result.

The prerequisites for this implementation include the installation of the requisite Python packages, such as langchain, langchain-community, and a model provider library like langchain-openai. Furthermore, a valid API key for the chosen language model must be configured in the local environment for authentication.

```
import os
import asyncio
from typing import Optional

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import Runnable, RunnableParallel,
RunnablePassthrough

# --- Configuration ---
# Ensure your API key environment variable is set (e.g.,
# OPENAI_API_KEY)
try:
    llm: Optional[ChatOpenAI] = ChatOpenAI(model="gpt-4o-mini",
    temperature=0.7)

except Exception as e:
    print(f"Error initializing language model: {e}")
    llm = None

# --- Define Independent Chains ---
# These three chains represent distinct tasks that can be executed in
# parallel.

summarize_chain: Runnable = (
    ChatPromptTemplate.from_messages([
        ("system", "Summarize the following topic concisely:"),
        ("user", "{topic}")
    ])
    | llm
    | StrOutputParser()
)

questions_chain: Runnable = (
    ChatPromptTemplate.from_messages([
```

```

        ("system", "Generate three interesting questions about the
following topic:"),
        ("user", "{topic}")
    ])
    | llm
    | StrOutputParser()
)

terms_chain: Runnable = (
    ChatPromptTemplate.from_messages([
        ("system", "Identify 5-10 key terms from the following topic,
separated by commas:"),
        ("user", "{topic}")
    ])
    | llm
    | StrOutputParser()
)

# --- Build the Parallel + Synthesis Chain ---

# 1. Define the block of tasks to run in parallel. The results of
these,
# along with the original topic, will be fed into the next step.
map_chain = RunnableParallel(
    {
        "summary": summarize_chain,
        "questions": questions_chain,
        "key_terms": terms_chain,
        "topic": RunnablePassthrough(), # Pass the original topic
through
    }
)

# 2. Define the final synthesis prompt which will combine the
parallel results.
synthesis_prompt = ChatPromptTemplate.from_messages([
    ("system", """"Based on the following information:
Summary: {summary}
Related Questions: {questions}
Key Terms: {key_terms}
Synthesize a comprehensive answer."""),
    ("user", "Original topic: {topic}")
])

# 3. Construct the full chain by piping the parallel results directly
# into the synthesis prompt, followed by the LLM and output
parser.

```

```

full_parallel_chain = map_chain | synthesis_prompt | llm |
StrOutputParser()

# --- Run the Chain ---
async def run_parallel_example(topic: str) -> None:
    """
    Asynchronously invokes the parallel processing chain with a
    specific topic
    and prints the synthesized result.

    Args:
        topic: The input topic to be processed by the LangChain
        chains.
    """
    if not llm:
        print("LLM not initialized. Cannot run example.")
        return

    print(f"\n--- Running Parallel LangChain Example for Topic:
    '{topic}' ---")
    try:
        # The input to `ainvoke` is the single 'topic' string,
        # then passed to each runnable in the `map_chain`.
        response = await full_parallel_chain.ainvoke(topic)
        print("\n--- Final Response ---")
        print(response)
    except Exception as e:
        print(f"\nAn error occurred during chain execution: {e}")

if __name__ == "__main__":
    test_topic = "The history of space exploration"
    # In Python 3.7+, asyncio.run is the standard way to run an async
    function.
    asyncio.run(run_parallel_example(test_topic))

```

The provided Python code implements a LangChain application designed for processing a given topic efficiently by leveraging parallel execution. Note that asyncio provides concurrency, not parallelism. It achieves this on a single thread by using an event loop that intelligently switches between tasks when one is idle (e.g., waiting for a network request). This creates the effect of multiple tasks progressing at once, but the code itself is still being executed by only one thread, constrained by Python's Global Interpreter Lock (GIL).

The code begins by importing essential modules from `langchain_openai` and `langchain_core`, including components for language models, prompts, output parsing, and runnable structures. The code attempts to initialize a `ChatOpenAI` instance, specifically using the "gpt-4o-mini" model, with a specified temperature for controlling creativity. A try-except block is used for robustness during the language model initialization. Three independent `LangChain` "chains" are then defined, each designed to perform a distinct task on the input topic. The first chain is for summarizing the topic concisely, using a system message and a user message containing the topic placeholder. The second chain is configured to generate three interesting questions related to the topic. The third chain is set up to identify between 5 and 10 key terms from the input topic, requesting them to be comma-separated. Each of these independent chains consists of a `ChatPromptTemplate` tailored to its specific task, followed by the initialized language model and a `StrOutputParser` to format the output as a string.

A `RunnableParallel` block is then constructed to bundle these three chains, allowing them to execute simultaneously. This parallel runnable also includes a `RunnablePassthrough` to ensure the original input topic is available for subsequent steps. A separate `ChatPromptTemplate` is defined for the final synthesis step, taking the summary, questions, key terms, and the original topic as input to generate a comprehensive answer. The full end-to-end processing chain, named `full_parallel_chain`, is created by sequencing the `map_chain` (the parallel block) into the synthesis prompt, followed by the language model and the output parser. An asynchronous function `run_parallel_example` is provided to demonstrate how to invoke this `full_parallel_chain`. This function takes the topic as input and uses `invoke` to run the asynchronous chain. Finally, the standard Python `if __name__ == "__main__":` block shows how to execute the `run_parallel_example` with a sample topic, in this case, "The history of space exploration", using `asyncio.run` to manage the asynchronous execution.

In essence, this code sets up a workflow where multiple LLM calls (for summarizing, questions, and terms) happen at the same time for a given topic, and their results are then combined by a final LLM call. This showcases the core idea of parallelization in an agentic workflow using `LangChain`.

Hands-On Code Example (Google ADK)

Okay, let's now turn our attention to a concrete example illustrating these concepts within the Google ADK framework. We'll examine how the ADK primitives, such as

ParallelAgent and SequentialAgent, can be applied to build an agent flow that leverages concurrent execution for improved efficiency.

```
from google.adk.agents import LlmAgent, ParallelAgent,
SequentialAgent
from google.adk.tools import google_search
GEMINI_MODEL="gemini-2.0-flash"

# --- 1. Define Researcher Sub-Agents (to run in parallel) ---

# Researcher 1: Renewable Energy
researcher_agent_1 = LlmAgent(
    name="RenewableEnergyResearcher",
    model=GEMINI_MODEL,
    instruction="""You are an AI Research Assistant specializing in
energy.
Research the latest advancements in 'renewable energy sources'.
Use the Google Search tool provided.
Summarize your key findings concisely (1-2 sentences).
Output *only* the summary.
""",
    description="Researches renewable energy sources.",
    tools=[google_search],
    # Store result in state for the merger agent
    output_key="renewable_energy_result"
)

# Researcher 2: Electric Vehicles
researcher_agent_2 = LlmAgent(
    name="EVResearcher",
    model=GEMINI_MODEL,
    instruction="""You are an AI Research Assistant specializing in
transportation.
Research the latest developments in 'electric vehicle technology'.
Use the Google Search tool provided.
Summarize your key findings concisely (1-2 sentences).
Output *only* the summary.
""",
    description="Researches electric vehicle technology.",
    tools=[google_search],
    # Store result in state for the merger agent
    output_key="ev_technology_result"
)

# Researcher 3: Carbon Capture
researcher_agent_3 = LlmAgent(
```



```

        name="CarbonCaptureResearcher",
        model=GEMINI_MODEL,
        instruction="""You are an AI Research Assistant specializing in
climate solutions.
Research the current state of 'carbon capture methods'.
Use the Google Search tool provided.
Summarize your key findings concisely (1-2 sentences).
Output *only* the summary.
""",
        description="Researches carbon capture methods.",
        tools=[google_search],
        # Store result in state for the merger agent
        output_key="carbon_capture_result"
    )

# --- 2. Create the ParallelAgent (Runs researchers concurrently) ---
# This agent orchestrates the concurrent execution of the
researchers.
# It finishes once all researchers have completed and stored their
results in state.
parallel_research_agent = ParallelAgent(
    name="ParallelWebResearchAgent",
    sub_agents=[researcher_agent_1, researcher_agent_2,
researcher_agent_3],
    description="Runs multiple research agents in parallel to gather
information."
)

# --- 3. Define the Merger Agent (Runs *after* the parallel agents)
---
# This agent takes the results stored in the session state by the
parallel agents
# and synthesizes them into a single, structured response with
attributions.
merger_agent = LlmAgent(
    name="SynthesisAgent",
    model=GEMINI_MODEL, # Or potentially a more powerful model if
needed for synthesis
    instruction="""You are an AI Assistant responsible for combining
research findings into a structured report.
Your primary task is to synthesize the following research summaries,
clearly attributing findings to their source areas. Structure your
response using headings for each topic. Ensure the report is coherent
and integrates the key points smoothly.

**Crucially: Your entire response MUST be grounded *exclusively* on
the information provided in the 'Input Summaries' below. Do NOT add

```

```

any external knowledge, facts, or details not present in these
specific summaries.**

**Input Summaries:**

*   **Renewable Energy:**
    {renewable_energy_result}
*   **Electric Vehicles:**
    {ev_technology_result}
*   **Carbon Capture:**
    {carbon_capture_result}

**Output Format:**

## Summary of Recent Sustainable Technology Advancements

### Renewable Energy Findings
(Based on RenewableEnergyResearcher's findings)
[Synthesize and elaborate *only* on the renewable energy input
summary provided above.]

### Electric Vehicle Findings
(Based on EVResearcher's findings)
[Synthesize and elaborate *only* on the EV input summary provided
above.]

### Carbon Capture Findings
(Based on CarbonCaptureResearcher's findings)
[Synthesize and elaborate *only* on the carbon capture input summary
provided above.]

### Overall Conclusion
[Provide a brief (1-2 sentence) concluding statement that connects
*only* the findings presented above.]

Output *only* the structured report following this format. Do not
include introductory or concluding phrases outside this structure,
and strictly adhere to using only the provided input summary content.
"""
    description="Combines research findings from parallel agents into
a structured, cited report, strictly grounded on provided inputs.",
    # No tools needed for merging
    # No output_key needed here, as its direct response is the final
output of the sequence
)

# --- 4. Create the SequentialAgent (Orchestrates the overall flow)

```

```

---
# This is the main agent that will be run. It first executes the
ParallelAgent
# to populate the state, and then executes the MergerAgent to produce
the final output.
sequential_pipeline_agent = SequentialAgent(
    name="ResearchAndSynthesisPipeline",
    # Run parallel research first, then merge
    sub_agents=[parallel_research_agent, merger_agent],
    description="Coordinates parallel research and synthesizes the
results."
)
root_agent = sequential_pipeline_agent

```

This code defines a multi-agent system used to research and synthesize information on sustainable technology advancements. It sets up three LlmAgent instances to act as specialized researchers. ResearcherAgent_1 focuses on renewable energy sources, ResearcherAgent_2 researches electric vehicle technology, and ResearcherAgent_3 investigates carbon capture methods. Each researcher agent is configured to use a GEMINI_MODEL and the google_search tool. They are instructed to summarize their findings concisely (1-2 sentences) and store these summaries in the session state using output_key.

A ParallelAgent named ParallelWebResearchAgent is then created to run these three researcher agents concurrently. This allows the research to be conducted in parallel, potentially saving time. The ParallelAgent completes its execution once all its sub-agents (the researchers) have finished and populated the state.

Next, a MergerAgent (also an LlmAgent) is defined to synthesize the research results. This agent takes the summaries stored in the session state by the parallel researchers as input. Its instruction emphasizes that the output must be strictly based only on the provided input summaries, prohibiting the addition of external knowledge. The MergerAgent is designed to structure the combined findings into a report with headings for each topic and a brief overall conclusion.

Finally, a SequentialAgent named ResearchAndSynthesisPipeline is created to orchestrate the entire workflow. As the primary controller, this main agent first executes the ParallelAgent to perform the research. Once the ParallelAgent is complete, the SequentialAgent then executes the MergerAgent to synthesize the collected information. The sequential_pipeline_agent is set as the root_agent, representing the entry point for running this multi-agent system. The overall process

is designed to efficiently gather information from multiple sources in parallel and then combine it into a single, structured report.

At a Glance

What: Many agentic workflows involve multiple sub-tasks that must be completed to achieve a final goal. A purely sequential execution, where each task waits for the previous one to finish, is often inefficient and slow. This latency becomes a significant bottleneck when tasks depend on external I/O operations, such as calling different APIs or querying multiple databases. Without a mechanism for concurrent execution, the total processing time is the sum of all individual task durations, hindering the system's overall performance and responsiveness.

Why: The Parallelization pattern provides a standardized solution by enabling the simultaneous execution of independent tasks. It works by identifying components of a workflow, like tool usages or LLM calls, that do not rely on each other's immediate outputs. Agentic frameworks like LangChain and the Google ADK provide built-in constructs to define and manage these concurrent operations. For instance, a main process can invoke several sub-tasks that run in parallel and wait for all of them to complete before proceeding to the next step. By running these independent tasks at the same time rather than one after another, this pattern drastically reduces the total execution time.

Rule of thumb: Use this pattern when a workflow contains multiple independent operations that can run simultaneously, such as fetching data from several APIs, processing different chunks of data, or generating multiple pieces of content for later synthesis.

Visual summary

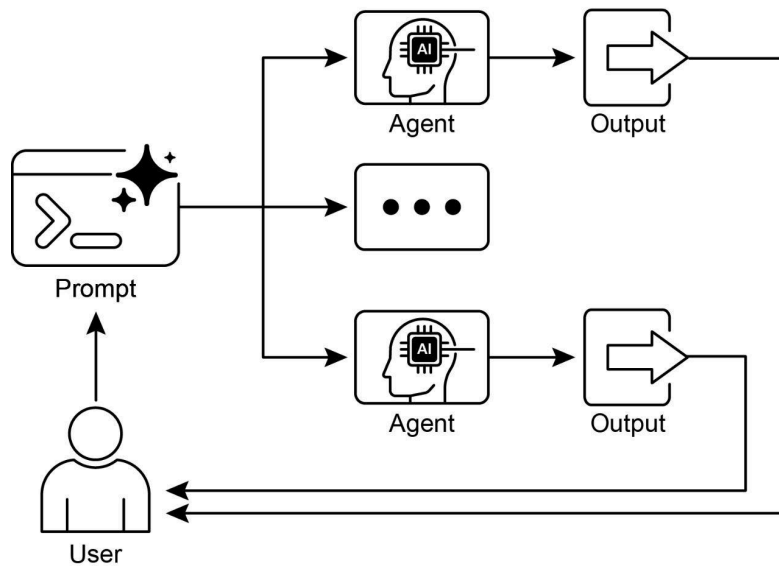


Fig.2: Parallelization design pattern

Key Takeaways

Here are the key takeaways:

- Parallelization is a pattern for executing independent tasks concurrently to improve efficiency.
- It is particularly useful when tasks involve waiting for external resources, such as API calls.
- The adoption of a concurrent or parallel architecture introduces substantial complexity and cost, impacting key development phases such as design, debugging, and system logging.
- Frameworks like LangChain and Google ADK provide built-in support for defining and managing parallel execution.
- In LangChain Expression Language (LCEL), RunnableParallel is a key construct for running multiple runnables side-by-side.

- Google ADK can facilitate parallel execution through LLM-Driven Delegation, where a Coordinator agent's LLM identifies independent sub-tasks and triggers their concurrent handling by specialized sub-agents.
- Parallelization helps reduce overall latency and makes agentic systems more responsive for complex tasks.

Conclusion

The parallelization pattern is a method for optimizing computational workflows by concurrently executing independent sub-tasks. This approach reduces overall latency, particularly in complex operations that involve multiple model inferences or calls to external services.

Frameworks provide distinct mechanisms for implementing this pattern. In LangChain, constructs like `RunnableParallel` are used to explicitly define and execute multiple processing chains simultaneously. In contrast, frameworks like the Google Agent Developer Kit (ADK) can achieve parallelization through multi-agent delegation, where a primary coordinator model assigns different sub-tasks to specialized agents that can operate concurrently.

By integrating parallel processing with sequential (chaining) and conditional (routing) control flows, it becomes possible to construct sophisticated, high-performance computational systems capable of efficiently managing diverse and complex tasks.

References

Here are some resources for further reading on the Parallelization pattern and related concepts:

1. LangChain Expression Language (LCEL) Documentation (Parallelism): <https://python.langchain.com/docs/concepts/lcel/>
2. Google Agent Developer Kit (ADK) Documentation (Multi-Agent Systems): <https://google.github.io/adk-docs/agents/multi-agents/>
3. Python asyncio Documentation: <https://docs.python.org/3/library/asyncio.html>

Chapter 4: Reflection

Reflection Pattern Overview

In the preceding chapters, we've explored fundamental agentic patterns: Chaining for sequential execution, Routing for dynamic path selection, and Parallelization for concurrent task execution. These patterns enable agents to perform complex tasks more efficiently and flexibly. However, even with sophisticated workflows, an agent's initial output or plan might not be optimal, accurate, or complete. This is where the **Reflection** pattern comes into play.

The Reflection pattern involves an agent evaluating its own work, output, or internal state and using that evaluation to improve its performance or refine its response. It's a form of self-correction or self-improvement, allowing the agent to iteratively refine its output or adjust its approach based on feedback, internal critique, or comparison against desired criteria. Reflection can occasionally be facilitated by a separate agent whose specific role is to analyze the output of an initial agent.

Unlike a simple sequential chain where output is passed directly to the next step, or routing which chooses a path, reflection introduces a feedback loop. The agent doesn't just produce an output; it then examines that output (or the process that generated it), identifies potential issues or areas for improvement, and uses those insights to generate a better version or modify its future actions.

The process typically involves:

1. **Execution:** The agent performs a task or generates an initial output.
2. **Evaluation/Critique:** The agent (often using another LLM call or a set of rules) analyzes the result from the previous step. This evaluation might check for factual accuracy, coherence, style, completeness, adherence to instructions, or other relevant criteria.
3. **Reflection/Refinement:** Based on the critique, the agent determines how to improve. This might involve generating a refined output, adjusting parameters for a subsequent step, or even modifying the overall plan.
4. **Iteration (Optional but common):** The refined output or adjusted approach can then be executed, and the reflection process can repeat until a satisfactory result is achieved or a stopping condition is met.

A key and highly effective implementation of the Reflection pattern separates the process into two distinct logical roles: a Producer and a Critic. This is often called the "Generator-Critic" or "Producer-Reviewer" model. While a single agent can perform self-reflection, using two specialized agents (or two separate LLM calls with distinct system prompts) often yields more robust and unbiased results.

1. The Producer Agent: This agent's primary responsibility is to perform the initial execution of the task. It focuses entirely on generating the content, whether it's writing code, drafting a blog post, or creating a plan. It takes the initial prompt and produces the first version of the output.

2. The Critic Agent: This agent's sole purpose is to evaluate the output generated by the Producer. It is given a different set of instructions, often a distinct persona (e.g., "You are a senior software engineer," "You are a meticulous fact-checker"). The Critic's instructions guide it to analyze the Producer's work against specific criteria, such as factual accuracy, code quality, stylistic requirements, or completeness. It is designed to find flaws, suggest improvements, and provide structured feedback.

This separation of concerns is powerful because it prevents the "cognitive bias" of an agent reviewing its own work. The Critic agent approaches the output with a fresh perspective, dedicated entirely to finding errors and areas for improvement. The feedback from the Critic is then passed back to the Producer agent, which uses it as a guide to generate a new, refined version of the output. The provided LangChain and ADK code examples both implement this two-agent model: the LangChain example uses a specific "reflector_prompt" to create a critic persona, while the ADK example explicitly defines a producer and a reviewer agent.

Implementing reflection often requires structuring the agent's workflow to include these feedback loops. This can be achieved through iterative loops in code, or using frameworks that support state management and conditional transitions based on evaluation results. While a single step of evaluation and refinement can be implemented within either a LangChain/LangGraph, or ADK, or Crew.AI chain, true iterative reflection typically involves more complex orchestration.

The Reflection pattern is crucial for building agents that can produce high-quality outputs, handle nuanced tasks, and exhibit a degree of self-awareness and adaptability. It moves agents beyond simply executing instructions towards a more sophisticated form of problem-solving and content generation.

The intersection of reflection with goal setting and monitoring (see Chapter 11) is worth noticing. A goal provides the ultimate benchmark for the agent's self-evaluation, while monitoring tracks its progress. In a number of practical cases, Reflection then might act as the corrective engine, using monitored feedback to analyze deviations and adjust its strategy. This synergy transforms the agent from a passive executor into a purposeful system that adaptively works to achieve its objectives.

Furthermore, the effectiveness of the Reflection pattern is significantly enhanced when the LLM keeps a memory of the conversation (see Chapter 8). This conversational history provides crucial context for the evaluation phase, allowing the agent to assess its output not just in isolation, but against the backdrop of previous interactions, user feedback, and evolving goals. It enables the agent to learn from past critiques and avoid repeating errors. Without memory, each reflection is a self-contained event; with memory, reflection becomes a cumulative process where each cycle builds upon the last, leading to more intelligent and context-aware refinement.

Practical Applications & Use Cases

The Reflection pattern is valuable in scenarios where output quality, accuracy, or adherence to complex constraints is critical:

1. Creative Writing and Content Generation:

Refining generated text, stories, poems, or marketing copy.

- **Use Case:** An agent writing a blog post.
 - **Reflection:** Generate a draft, critique it for flow, tone, and clarity, then rewrite based on the critique. Repeat until the post meets quality standards.
 - **Benefit:** Produces more polished and effective content.

2. Code Generation and Debugging:

Writing code, identifying errors, and fixing them.

- **Use Case:** An agent writing a Python function.
 - **Reflection:** Write initial code, run tests or static analysis, identify errors or inefficiencies, then modify the code based on the findings.
 - **Benefit:** Generates more robust and functional code.

3. Complex Problem Solving:

Evaluating intermediate steps or proposed solutions in multi-step reasoning tasks.

- **Use Case:** An agent solving a logic puzzle.

- **Reflection:** Propose a step, evaluate if it leads closer to the solution or introduces contradictions, backtrack or choose a different step if needed.
- **Benefit:** Improves the agent's ability to navigate complex problem spaces.

4. Summarization and Information Synthesis:

Refining summaries for accuracy, completeness, and conciseness.

- **Use Case:** An agent summarizing a long document.
 - **Reflection:** Generate an initial summary, compare it against key points in the original document, refine the summary to include missing information or improve accuracy.
 - **Benefit:** Creates more accurate and comprehensive summaries.

5. Planning and Strategy:

Evaluating a proposed plan and identifying potential flaws or improvements.

- **Use Case:** An agent planning a series of actions to achieve a goal.
 - **Reflection:** Generate a plan, simulate its execution or evaluate its feasibility against constraints, revise the plan based on the evaluation.
 - **Benefit:** Develops more effective and realistic plans.

6. Conversational Agents:

Reviewing previous turns in a conversation to maintain context, correct misunderstandings, or improve response quality.

- **Use Case:** A customer support chatbot.
 - **Reflection:** After a user response, review the conversation history and the last generated message to ensure coherence and address the user's latest input accurately.
 - **Benefit:** Leads to more natural and effective conversations.

Reflection adds a layer of meta-cognition to agentic systems, enabling them to learn from their own outputs and processes, leading to more intelligent, reliable, and high-quality results.

Hands-On Code Example (LangChain)

The implementation of a complete, iterative reflection process necessitates mechanisms for state management and cyclical execution. While these are handled natively in graph-based frameworks like LangGraph or through custom procedural code, the fundamental principle of a single reflection cycle can be demonstrated effectively using the compositional syntax of LCEL (LangChain Expression Language).

This example implements a reflection loop using the Langchain library and OpenAI's GPT-4o model to iteratively generate and refine a Python function that calculates the

factorial of a number. The process starts with a task prompt, generates initial code, and then repeatedly reflects on the code based on critiques from a simulated senior software engineer role, refining the code in each iteration until the critique stage determines the code is perfect or a maximum number of iterations is reached. Finally, it prints the resulting refined code.

First, ensure you have the necessary libraries installed:

```
pip install langchain langchain-community langchain-openai
```

You will also need to set up your environment with your API key for the language model you choose (e.g., OpenAI, Google Gemini, Anthropic).

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.messages import SystemMessage, HumanMessage

# --- Configuration ---
# Load environment variables from .env file (for OPENAI_API_KEY)
load_dotenv()

# Check if the API key is set
if not os.getenv("OPENAI_API_KEY"):
    raise ValueError("OPENAI_API_KEY not found in .env file. Please
add it.")

# Initialize the Chat LLM. We use gpt-4o for better reasoning.
# A lower temperature is used for more deterministic outputs.
llm = ChatOpenAI(model="gpt-4o", temperature=0.1)

def run_reflection_loop():
    """
    Demonstrates a multi-step AI reflection loop to progressively
    improve a Python function.
    """
    # --- The Core Task ---
    task_prompt = """
    Your task is to create a Python function named
    `calculate_factorial`.
    This function should do the following:
    1. Accept a single integer `n` as input.
```

```

2. Calculate its factorial (n!).
3. Include a clear docstring explaining what the function does.
4. Handle edge cases: The factorial of 0 is 1.
5. Handle invalid input: Raise a ValueError if the input is a
negative number.
"""
# --- The Reflection Loop ---
max_iterations = 3
current_code = ""
# We will build a conversation history to provide context in each
step.
message_history = [HumanMessage(content=task_prompt)]

for i in range(max_iterations):
    print("\n" + "="*25 + f" REFLECTION LOOP: ITERATION {i + 1} "
+ "="*25)

    # --- 1. GENERATE / REFINE STAGE ---
    # In the first iteration, it generates. In subsequent
iterations, it refines.
    if i == 0:
        print("\n>>> STAGE 1: GENERATING initial code...")
        # The first message is just the task prompt.
        response = llm.invoke(message_history)
        current_code = response.content
    else:
        print("\n>>> STAGE 1: REFINING code based on previous
critique...")
        # The message history now contains the task,
        # the last code, and the last critique.
        # We instruct the model to apply the critiques.
        message_history.append(HumanMessage(content="Please refine
the code using the critiques provided."))
        response = llm.invoke(message_history)
        current_code = response.content

    print("\n--- Generated Code (v" + str(i + 1) + ") ---\n" +
current_code)
    message_history.append(response) # Add the generated code to
history

    # --- 2. REFLECT STAGE ---
    print("\n>>> STAGE 2: REFLECTING on the generated code...")

    # Create a specific prompt for the reflector agent.
    # This asks the model to act as a senior code reviewer.

```

```

reflector_prompt = [
    SystemMessage(content="""
        You are a senior software engineer and an expert
        in Python.
        Your role is to perform a meticulous code review.
        Critically evaluate the provided Python code based
        on the original task requirements.
        Look for bugs, style issues, missing edge cases,
        and areas for improvement.
        If the code is perfect and meets all requirements,
        respond with the single phrase 'CODE_IS_PERFECT'.
        Otherwise, provide a bulleted list of your critiques.
    """),
    HumanMessage(content=f"Original
Task:\n{task_prompt}\n\nCode to Review:\n{current_code}")
]

critique_response = llm.invoke(reflector_prompt)
critique = critique_response.content

# --- 3. STOPPING CONDITION ---
if "CODE_IS_PERFECT" in critique:
    print("\n--- Critique ---\nNo further critiques found. The
code is satisfactory.")
    break

print("\n--- Critique ---\n" + critique)
# Add the critique to the history for the next refinement
loop.
    message_history.append(HumanMessage(content=f"Critique of the
previous code:\n{critique}"))

print("\n" + "="*30 + " FINAL RESULT " + "="*30)
print("\nFinal refined code after the reflection process:\n")
print(current_code)

if __name__ == "__main__":
    run_reflection_loop()

```

The code begins by setting up the environment, loading API keys, and initializing a powerful language model like GPT-4o with a low temperature for focused outputs. The core task is defined by a prompt asking for a Python function to calculate the factorial of a number, including specific requirements for docstrings, edge cases (factorial of 0), and error handling for negative input. The `run_reflection_loop` function orchestrates the iterative refinement process. Within the loop, in the first iteration, the

language model generates initial code based on the task prompt. In subsequent iterations, it refines the code based on critiques from the previous step. A separate "reflector" role, also played by the language model but with a different system prompt, acts as a senior software engineer to critique the generated code against the original task requirements. This critique is provided as a bulleted list of issues or the phrase 'CODE_IS_PERFECT' if no issues are found. The loop continues until the critique indicates the code is perfect or a maximum number of iterations is reached. The conversation history is maintained and passed to the language model in each step to provide context for both generation/refinement and reflection stages. Finally, the script prints the last generated code version after the loop concludes.

Hands-On Code Example (ADK)

Let's now look at a conceptual code example implemented using the Google ADK. Specifically, the code showcases this by employing a Generator-Critic structure, where one component (the Generator) produces an initial result or plan, and another component (the Critic) provides critical feedback or a critique, guiding the Generator towards a more refined or accurate final output.

```
from google.adk.agents import SequentialAgent, LlmAgent

# The first agent generates the initial draft.
generator = LlmAgent(
    name="DraftWriter",
    description="Generates initial draft content on a given subject.",
    instruction="Write a short, informative paragraph about the user's subject.",
    output_key="draft_text" # The output is saved to this state key.
)

# The second agent critiques the draft from the first agent.
reviewer = LlmAgent(
    name="FactChecker",
    description="Reviews a given text for factual accuracy and provides a structured critique.",
    instruction="""
You are a meticulous fact-checker.
1. Read the text provided in the state key 'draft_text'.
2. Carefully verify the factual accuracy of all claims.
3. Your final output must be a dictionary containing two keys:
   - "status": A string, either "ACCURATE" or "INACCURATE".
   - "reasoning": A string providing a clear explanation for your status, citing specific issues if any are found.
    """
)
```

```

    """
    output_key="review_output" # The structured dictionary is saved
    here.
    )

# The SequentialAgent ensures the generator runs before the reviewer.
review_pipeline = SequentialAgent(
    name="WriteAndReview_Pipeline",
    sub_agents=[generator, reviewer]
)

# Execution Flow:
# 1. generator runs -> saves its paragraph to state['draft_text'].
# 2. reviewer runs -> reads state['draft_text'] and saves its
    dictionary output to state['review_output'].

```

This code demonstrates the use of a sequential agent pipeline in Google ADK for generating and reviewing text. It defines two LlmAgent instances: generator and reviewer. The generator agent is designed to create an initial draft paragraph on a given subject. It is instructed to write a short and informative piece and saves its output to the state key draft_text. The reviewer agent acts as a fact-checker for the text produced by the generator. It is instructed to read the text from draft_text and verify its factual accuracy. The reviewer's output is a structured dictionary with two keys: status and reasoning. status indicates if the text is "ACCURATE" or "INACCURATE", while reasoning provides an explanation for the status. This dictionary is saved to the state key review_output. A SequentialAgent named review_pipeline is created to manage the execution order of the two agents. It ensures that the generator runs first, followed by the reviewer. The overall execution flow is that the generator produces text, which is then saved to the state. Subsequently, the reviewer reads this text from the state, performs its fact-checking, and saves its findings (the status and reasoning) back to the state. This pipeline allows for a structured process of content creation and review using separate agents.**Note:** An alternative implementation utilizing ADK's LoopAgent is also available for those interested.

Before concluding, it's important to consider that while the Reflection pattern significantly enhances output quality, it comes with important trade-offs. The iterative process, though powerful, can lead to higher costs and latency, since every refinement loop may require a new LLM call, making it suboptimal for time-sensitive applications. Furthermore, the pattern is memory-intensive; with each iteration, the conversational history expands, including the initial output, critique, and subsequent refinements.

At Glance

What: An agent's initial output is often suboptimal, suffering from inaccuracies, incompleteness, or a failure to meet complex requirements. Basic agentic workflows lack a built-in process for the agent to recognize and fix its own errors. This is solved by having the agent evaluate its own work or, more robustly, by introducing a separate logical agent to act as a critic, preventing the initial response from being the final one regardless of quality.

Why: The Reflection pattern offers a solution by introducing a mechanism for self-correction and refinement. It establishes a feedback loop where a "producer" agent generates an output, and then a "critic" agent (or the producer itself) evaluates it against predefined criteria. This critique is then used to generate an improved version. This iterative process of generation, evaluation, and refinement progressively enhances the quality of the final result, leading to more accurate, coherent, and reliable outcomes.

Rule of thumb: Use the Reflection pattern when the quality, accuracy, and detail of the final output are more important than speed and cost. It is particularly effective for tasks like generating polished long-form content, writing and debugging code, and creating detailed plans. Employ a separate critic agent when tasks require high objectivity or specialized evaluation that a generalist producer agent might miss.

Visual summary

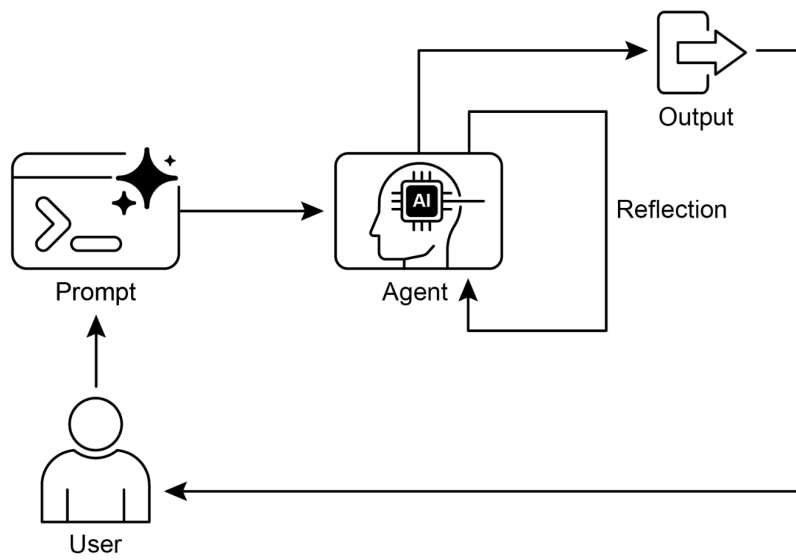


Fig. 1: Reflection design pattern, self-reflection

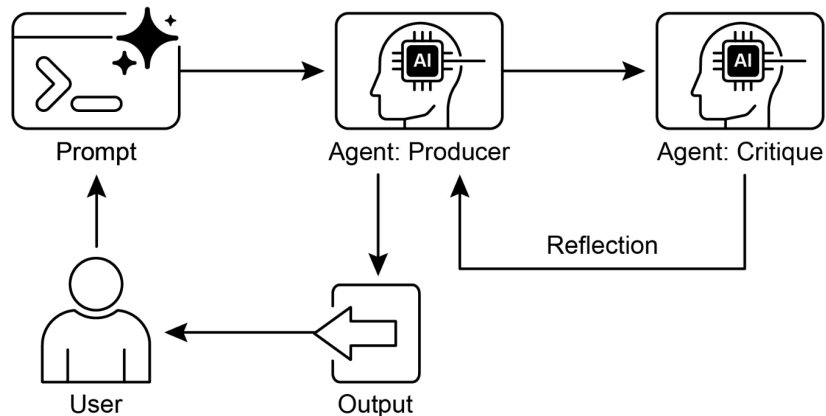


Fig.2: Reflection design pattern, producer and critique agent

Key Takeaways

- The primary advantage of the Reflection pattern is its ability to iteratively self-correct and refine outputs, leading to significantly higher quality, accuracy, and adherence to complex instructions.
- It involves a feedback loop of execution, evaluation/critique, and refinement. Reflection is essential for tasks requiring high-quality, accurate, or nuanced outputs.
- A powerful implementation is the Producer-Critic model, where a separate agent (or prompted role) evaluates the initial output. This separation of concerns enhances objectivity and allows for more specialized, structured feedback.

- However, these benefits come at the cost of increased latency and computational expense, along with a higher risk of exceeding the model's context window or being throttled by API services.
- While full iterative reflection often requires stateful workflows (like LangGraph), a single reflection step can be implemented in LangChain using LCEL to pass output for critique and subsequent refinement.
- Google ADK can facilitate reflection through sequential workflows where one agent's output is critiqued by another agent, allowing for subsequent refinement steps.
- This pattern enables agents to perform self-correction and enhance their performance over time.

Conclusion

The reflection pattern provides a crucial mechanism for self-correction within an agent's workflow, enabling iterative improvement beyond a single-pass execution. This is achieved by creating a loop where the system generates an output, evaluates it against specific criteria, and then uses that evaluation to produce a refined result. This evaluation can be performed by the agent itself (self-reflection) or, often more effectively, by a distinct critic agent, which represents a key architectural choice within the pattern.

While a fully autonomous, multi-step reflection process requires a robust architecture for state management, its core principle is effectively demonstrated in a single generate-critique-refine cycle. As a control structure, reflection can be integrated with other foundational patterns to construct more robust and functionally complex agentic systems.

References

Here are some resources for further reading on the Reflection pattern and related concepts:

1. Training Language Models to Self-Correct via Reinforcement Learning, <https://arxiv.org/abs/2409.12917>
2. LangChain Expression Language (LCEL) Documentation: <https://python.langchain.com/docs/introduction/>
3. LangGraph Documentation: <https://www.langchain.com/langgraph>

4. Google Agent Developer Kit (ADK) Documentation (Multi-Agent Systems):
<https://google.github.io/adk-docs/agents/multi-agents/>

Chapter 5: Tool Use (Function Calling)

Tool Use Pattern Overview

So far, we've discussed agentic patterns that primarily involve orchestrating interactions between language models and managing the flow of information within the agent's internal workflow (Chaining, Routing, Parallelization, Reflection). However, for agents to be truly useful and interact with the real world or external systems, they need the ability to use Tools.

The Tool Use pattern, often implemented through a mechanism called Function Calling, enables an agent to interact with external APIs, databases, services, or even execute code. It allows the LLM at the core of the agent to decide when and how to use a specific external function based on the user's request or the current state of the task.

The process typically involves:

1. **Tool Definition:** External functions or capabilities are defined and described to the LLM. This description includes the function's purpose, its name, and the parameters it accepts, along with their types and descriptions.
2. **LLM Decision:** The LLM receives the user's request and the available tool definitions. Based on its understanding of the request and the tools, the LLM decides if calling one or more tools is necessary to fulfill the request.
3. **Function Call Generation:** If the LLM decides to use a tool, it generates a structured output (often a JSON object) that specifies the name of the tool to call and the arguments (parameters) to pass to it, extracted from the user's request.
4. **Tool Execution:** The agentic framework or orchestration layer intercepts this structured output. It identifies the requested tool and executes the actual external function with the provided arguments.
5. **Observation/Result:** The output or result from the tool execution is returned to the agent.
6. **LLM Processing (Optional but common):** The LLM receives the tool's output as context and uses it to formulate a final response to the user or decide on the next step in the workflow (which might involve calling another tool, reflecting, or providing a final answer).

This pattern is fundamental because it breaks the limitations of the LLM's training data and allows it to access up-to-date information, perform calculations it can't do internally, interact with user-specific data, or trigger real-world actions. Function

calling is the technical mechanism that bridges the gap between the LLM's reasoning capabilities and the vast array of external functionalities available.

While "function calling" aptly describes invoking specific, predefined code functions, it's useful to consider the more expansive concept of "tool calling." This broader term acknowledges that an agent's capabilities can extend far beyond simple function execution. A "tool" can be a traditional function, but it can also be a complex API endpoint, a request to a database, or even an instruction directed at another specialized agent. This perspective allows us to envision more sophisticated systems where, for instance, a primary agent might delegate a complex data analysis task to a dedicated "analyst agent" or query an external knowledge base through its API. Thinking in terms of "tool calling" better captures the full potential of agents to act as orchestrators across a diverse ecosystem of digital resources and other intelligent entities.

Frameworks like LangChain, LangGraph, and Google Agent Developer Kit (ADK) provide robust support for defining tools and integrating them into agent workflows, often leveraging the native function calling capabilities of modern LLMs like those in the Gemini or OpenAI series. On the "canvas" of these frameworks, you define the tools and then configure agents (typically LLM Agents) to be aware of and capable of using these tools.

Tool Use is a cornerstone pattern for building powerful, interactive, and externally aware agents.

Practical Applications & Use Cases

The Tool Use pattern is applicable in virtually any scenario where an agent needs to go beyond generating text to perform an action or retrieve specific, dynamic information:

1. Information Retrieval from External Sources:

Accessing real-time data or information that is not present in the LLM's training data.

- **Use Case:** A weather agent.
 - **Tool:** A weather API that takes a location and returns the current weather conditions.
 - **Agent Flow:** User asks, "What's the weather in London?", LLM identifies the need for the weather tool, calls the tool with "London", tool returns data, LLM formats the data into a user-friendly response.

2. Interacting with Databases and APIs:

Performing queries, updates, or other operations on structured data.

- **Use Case:** An e-commerce agent.
 - **Tools:** API calls to check product inventory, get order status, or process payments.
 - **Agent Flow:** User asks "Is product X in stock?", LLM calls the inventory API, tool returns stock count, LLM tells the user the stock status.

3. Performing Calculations and Data Analysis:

Using external calculators, data analysis libraries, or statistical tools.

- **Use Case:** A financial agent.
 - **Tools:** A calculator function, a stock market data API, a spreadsheet tool.
 - **Agent Flow:** User asks "What's the current price of AAPL and calculate the potential profit if I bought 100 shares at \$150?", LLM calls stock API, gets current price, then calls calculator tool, gets result, formats response.

4. Sending Communications:

Sending emails, messages, or making API calls to external communication services.

- **Use Case:** A personal assistant agent.
 - **Tool:** An email sending API.
 - **Agent Flow:** User says, "Send an email to John about the meeting tomorrow.", LLM calls an email tool with the recipient, subject, and body extracted from the request.

5. Executing Code:

Running code snippets in a safe environment to perform specific tasks.

- **Use Case:** A coding assistant agent.
 - **Tool:** A code interpreter.
 - **Agent Flow:** User provides a Python snippet and asks, "What does this code do?", LLM uses the interpreter tool to run the code and analyze its output.

6. Controlling Other Systems or Devices:

Interacting with smart home devices, IoT platforms, or other connected systems.

- **Use Case:** A smart home agent.
 - **Tool:** An API to control smart lights.
 - **Agent Flow:** User says, "Turn off the living room lights." LLM calls the smart home tool with the command and target device.

Tool Use is what transforms a language model from a text generator into an agent capable of sensing, reasoning, and acting in the digital or physical world (see Fig. 1)

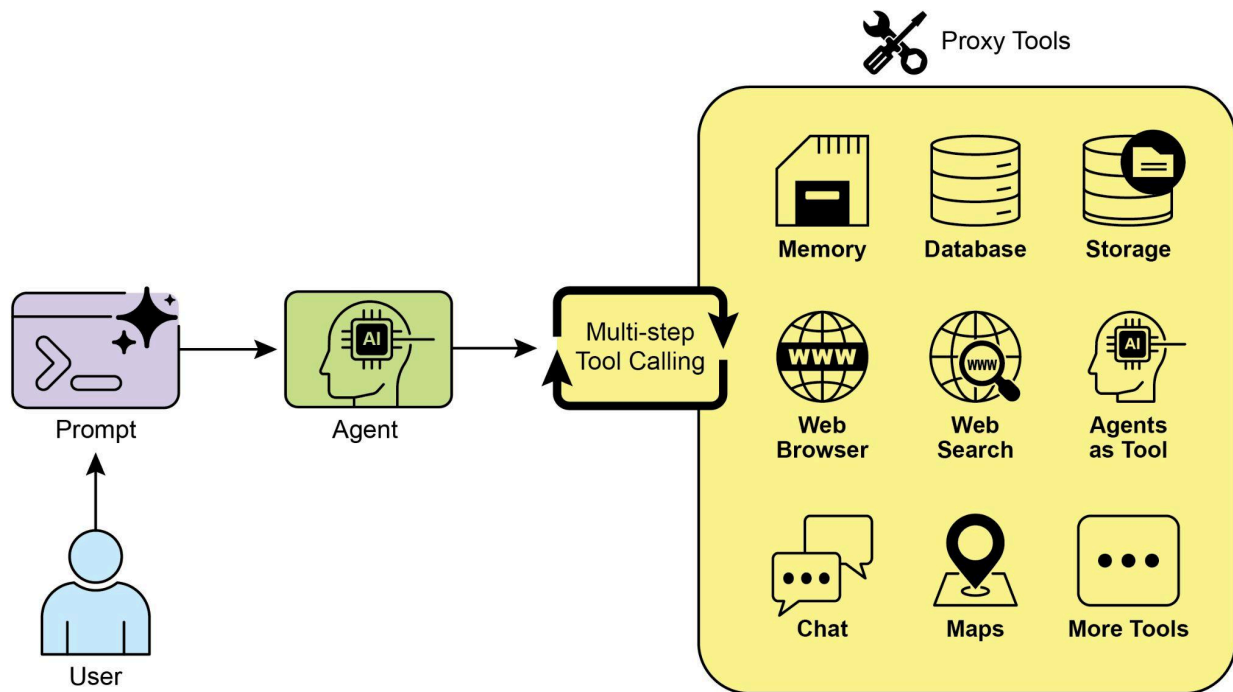


Fig.1: Some examples of an Agent using Tools

Hands-On Code Example (LangChain)

The implementation of tool use within the LangChain framework is a two-stage process. Initially, one or more tools are defined, typically by encapsulating existing Python functions or other runnable components. Subsequently, these tools are bound to a language model, thereby granting the model the capability to generate a structured tool-use request when it determines that an external function call is required to fulfill a user's query.

The following implementation will demonstrate this principle by first defining a simple function to simulate an information retrieval tool. Following this, an agent will be constructed and configured to leverage this tool in response to user input. The execution of this example requires the installation of the core LangChain libraries and a model-specific provider package. Furthermore, proper authentication with the

selected language model service, typically via an API key configured in the local environment, is a necessary prerequisite.

```
import os, getpass
import asyncio
import nest_asyncio
from typing import List
from dotenv import load_dotenv
import logging

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.tools import tool as langchain_tool
from langchain.agents import create_tool_calling_agent, AgentExecutor

# UNCOMMENT
# Prompt the user securely and set API keys as an environment
variables
os.environ["GOOGLE_API_KEY"] = getpass.getpass("Enter your Google API
key: ")
os.environ["OPENAI_API_KEY"] = getpass.getpass("Enter your OpenAI API
key: ")

try:
    # A model with function/tool calling capabilities is required.
    llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash",
temperature=0)
    print(f"✅ Language model initialized: {llm.model}")
except Exception as e:
    print(f"❌ Error initializing language model: {e}")
    llm = None

# --- Define a Tool ---
@langchain_tool
def search_information(query: str) -> str:
    """
    Provides factual information on a given topic. Use this tool to
    find answers to phrases
    like 'capital of France' or 'weather in London?'.
    """
    print(f"\n--- 🛠️ Tool Called: search_information with query:
'{query}' ---")
    # Simulate a search tool with a dictionary of predefined results.
    simulated_results = {
        "weather in london": "The weather in London is currently cloudy
with a temperature of 15°C.",
```

```

        "capital of france": "The capital of France is Paris.",
        "population of earth": "The estimated population of Earth is
around 8 billion people.",
        "tallest mountain": "Mount Everest is the tallest mountain
above sea level.",
        "default": f"Simulated search result for '{query}': No specific
information found, but the topic seems interesting."
    }
    result = simulated_results.get(query.lower(),
simulated_results["default"])
    print(f"--- TOOL RESULT: {result} ---")
    return result

tools = [search_information]

# --- Create a Tool-Calling Agent ---
if llm:
    # This prompt template requires an `agent_scratchpad` placeholder
    for the agent's internal steps.
    agent_prompt = ChatPromptTemplate.from_messages([
        ("system", "You are a helpful assistant."),
        ("human", "{input}"),
        ("placeholder", "{agent_scratchpad}"),
    ])

    # Create the agent, binding the LLM, tools, and prompt together.
    agent = create_tool_calling_agent(llm, tools, agent_prompt)

    # AgentExecutor is the runtime that invokes the agent and executes
    the chosen tools.
    # The 'tools' argument is not needed here as they are already bound
    to the agent.
    agent_executor = AgentExecutor(agent=agent, verbose=True,
tools=tools)

async def run_agent_with_tool(query: str):
    """Invokes the agent executor with a query and prints the final
response."""
    print(f"\n--- 🏃 Running Agent with Query: '{query}' ---")
    try:
        response = await agent_executor.ainvoke({"input": query})
        print(f"\n--- ✅ Final Agent Response ---")
        print(response["output"])
    except Exception as e:
        print(f"\n🛑 An error occurred during agent execution: {e}")

async def main():

```

```

    """Runs all agent queries concurrently."""
    tasks = [
        run_agent_with_tool("What is the capital of France?"),
        run_agent_with_tool("What's the weather like in London?"),
        run_agent_with_tool("Tell me something about dogs.") # Should
trigger the default tool response
    ]
    await asyncio.gather(*tasks)

nest_asyncio.apply()
asyncio.run(main())

```

The code sets up a tool-calling agent using the LangChain library and the Google Gemini model. It defines a `search_information` tool that simulates providing factual answers to specific queries. The tool has predefined responses for "weather in london," "capital of france," and "population of earth," and a default response for other queries. A `ChatGoogleGenerativeAI` model is initialized, ensuring it has tool-calling capabilities. A `ChatPromptTemplate` is created to guide the agent's interaction. The `create_tool_calling_agent` function is used to combine the language model, tools, and prompt into an agent. An `AgentExecutor` is then set up to manage the agent's execution and tool invocation. The `run_agent_with_tool` asynchronous function is defined to invoke the agent with a given query and print the result. The main asynchronous function prepares multiple queries to be run concurrently. These queries are designed to test both the specific and default responses of the `search_information` tool. Finally, the `asyncio.run(main())` call executes all the agent tasks. The code includes checks for successful LLM initialization before proceeding with agent setup and execution.

Hands-On Code Example (CrewAI)

This code provides a practical example of how to implement function calling (Tools) within the CrewAI framework. It sets up a simple scenario where an agent is equipped with a tool to look up information. The example specifically demonstrates fetching a simulated stock price using this agent and tool.

```

# pip install crewai langchain-openai

import os
from crewai import Agent, Task, Crew
from crewai.tools import tool
import logging

```

```

# --- Best Practice: Configure Logging ---
# A basic logging setup helps in debugging and tracking the crew's
execution.
logging.basicConfig(level=logging.INFO, format='%(asctime)s -
%(levelname)s - %(message)s')

# --- Set up your API Key ---
# For production, it's recommended to use a more secure method for
key management
# like environment variables loaded at runtime or a secret manager.
#
# Set the environment variable for your chosen LLM provider (e.g.,
OPENAI_API_KEY)
# os.environ["OPENAI_API_KEY"] = "YOUR_API_KEY"
# os.environ["OPENAI_MODEL_NAME"] = "gpt-4o"

# --- 1. Refactored Tool: Returns Clean Data ---
# The tool now returns raw data (a float) or raises a standard Python
error.
# This makes it more reusable and forces the agent to handle outcomes
properly.
@tool("Stock Price Lookup Tool")
def get_stock_price(ticker: str) -> float:
    """
    Fetches the latest simulated stock price for a given stock ticker
symbol.
    Returns the price as a float. Raises a ValueError if the ticker is
not found.
    """
    logging.info(f"Tool Call: get_stock_price for ticker '{ticker}'")
    simulated_prices = {
        "AAPL": 178.15,
        "GOOGL": 1750.30,
        "MSFT": 425.50,
    }
    price = simulated_prices.get(ticker.upper())

    if price is not None:
        return price
    else:
        # Raising a specific error is better than returning a string.
        # The agent is equipped to handle exceptions and can decide on
the next action.
        raise ValueError(f"Simulated price for ticker
'{ticker.upper()}' not found.")

```

```

# --- 2. Define the Agent ---
# The agent definition remains the same, but it will now leverage the
improved tool.
financial_analyst_agent = Agent(
    role='Senior Financial Analyst',
    goal='Analyze stock data using provided tools and report key
prices.',
    backstory="You are an experienced financial analyst adept at using
data sources to find stock information. You provide clear, direct
answers.",
    verbose=True,
    tools=[get_stock_price],
    # Allowing delegation can be useful, but is not necessary for this
simple task.
    allow_delegation=False,
)

# --- 3. Refined Task: Clearer Instructions and Error Handling ---
# The task description is more specific and guides the agent on how
to react
# to both successful data retrieval and potential errors.
analyze_aapl_task = Task(
    description=(
        "What is the current simulated stock price for Apple (ticker:
AAPL)? "
        "Use the 'Stock Price Lookup Tool' to find it. "
        "If the ticker is not found, you must report that you were
unable to retrieve the price."
    ),
    expected_output=(
        "A single, clear sentence stating the simulated stock price for
AAPL. "
        "For example: 'The simulated stock price for AAPL is $178.15.' "
        "If the price cannot be found, state that clearly."
    ),
    agent=financial_analyst_agent,
)

# --- 4. Formulate the Crew ---
# The crew orchestrates how the agent and task work together.
financial_crew = Crew(
    agents=[financial_analyst_agent],
    tasks=[analyze_aapl_task],
    verbose=True # Set to False for less detailed logs in production
)

# --- 5. Run the Crew within a Main Execution Block ---

```

```

# Using a __name__ == "__main__": block is a standard Python best
practice.
def main():
    """Main function to run the crew."""
    # Check for API key before starting to avoid runtime errors.
    if not os.environ.get("OPENAI_API_KEY"):
        print("ERROR: The OPENAI_API_KEY environment variable is not
set.")
        print("Please set it before running the script.")
        return

    print("\n## Starting the Financial Crew...")
    print("-----")

    # The kickoff method starts the execution.
    result = financial_crew.kickoff()

    print("\n-----")
    print("## Crew execution finished.")
    print("\nFinal Result:\n", result)

if __name__ == "__main__":
    main()

```

This code demonstrates a simple application using the Crew.ai library to simulate a financial analysis task. It defines a custom tool, `get_stock_price`, that simulates looking up stock prices for predefined tickers. The tool is designed to return a floating-point number for valid tickers or raise a `ValueError` for invalid ones. A Crew.ai Agent named `financial_analyst_agent` is created with the role of a Senior Financial Analyst. This agent is given the `get_stock_price` tool to interact with. A Task is defined, `analyze_aapl_task`, specifically instructing the agent to find the simulated stock price for AAPL using the tool. The task description includes clear instructions on how to handle both success and failure cases when using the tool. A Crew is assembled, comprising the `financial_analyst_agent` and the `analyze_aapl_task`. The verbose setting is enabled for both the agent and the crew to provide detailed logging during execution. The main part of the script runs the crew's task using the `kickoff()` method within a standard `if __name__ == "__main__":` block. Before starting the crew, it checks if the `OPENAI_API_KEY` environment variable is set, which is required for the agent to function. The result of the crew's execution, which is the output of the task, is then printed to the console. The code also includes basic logging configuration for better tracking of the crew's actions and tool calls. It uses environment variables for API key management, though it notes that more secure methods are recommended for

production environments. In short, the core logic showcases how to define tools, agents, and tasks to create a collaborative workflow in Crew.ai.

Hands-on code (Google ADK)

The Google Agent Developer Kit (ADK) includes a library of natively integrated tools that can be directly incorporated into an agent's capabilities.

Google search: A primary example of such a component is the Google Search tool. This tool serves as a direct interface to the Google Search engine, equipping the agent with the functionality to perform web searches and retrieve external information.

```
from google.adk.agents import Agent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.adk.tools import google_search
from google.genai import types
import nest_asyncio
import asyncio

# Define variables required for Session setup and Agent execution
APP_NAME="Google Search_agent"
USER_ID="user1234"
SESSION_ID="1234"

# Define Agent with access to search tool
root_agent = ADKAgent(
    name="basic_search_agent",
    model="gemini-2.0-flash-exp",
    description="Agent to answer questions using Google Search.",
    instruction="I can answer your questions by searching the internet. Just ask me anything!",
    tools=[google_search] # Google Search is a pre-built tool to perform Google searches.
)

# Agent Interaction
async def call_agent(query):
    """
    Helper function to call the agent with a query.
    """

# Session and Runner
```

```

    session_service = InMemorySessionService()
    session = await session_service.create_session(app_name=APP_NAME,
user_id=USER_ID, session_id=SESSION_ID)
    runner = Runner(agent=root_agent, app_name=APP_NAME,
session_service=session_service)

    content = types.Content(role='user',
parts=[types.Part(text=query)])
    events = runner.run(user_id=USER_ID, session_id=SESSION_ID,
new_message=content)

    for event in events:
        if event.is_final_response():
            final_response = event.content.parts[0].text
            print("Agent Response: ", final_response)

nest_asyncio.apply()

asyncio.run(call_agent("what's the latest ai news?"))

```

This code demonstrates how to create and use a basic agent powered by the Google ADK for Python. The agent is designed to answer questions by utilizing Google Search as a tool. First, necessary libraries from IPython, google.adk, and google.genai are imported. Constants for the application name, user ID, and session ID are defined. An Agent instance named "basic_search_agent" is created with a description and instructions indicating its purpose. It's configured to use the Google Search tool, which is a pre-built tool provided by the ADK. An InMemorySessionService (see Chapter 8) is initialized to manage sessions for the agent. A new session is created for the specified application, user, and session IDs. A Runner is instantiated, linking the created agent with the session service. This runner is responsible for executing the agent's interactions within a session. A helper function call_agent is defined to simplify the process of sending a query to the agent and processing the response. Inside call_agent, the user's query is formatted as a types.Content object with the role 'user'. The runner.run method is called with the user ID, session ID, and the new message content. The runner.run method returns a list of events representing the agent's actions and responses. The code iterates through these events to find the final response. If an event is identified as the final response, the text content of that response is extracted. The extracted agent response is then printed to the console. Finally, the call_agent function is called with the query "what's the latest ai news?" to demonstrate the agent in action.

Code execution: The Google ADK features integrated components for specialized tasks, including an environment for dynamic code execution. The `built_in_code_execution` tool provides an agent with a sandboxed Python interpreter. This allows the model to write and run code to perform computational tasks, manipulate data structures, and execute procedural scripts. Such functionality is critical for addressing problems that require deterministic logic and precise calculations, which are outside the scope of probabilistic language generation alone.

```
import os, getpass
import asyncio
import nest_asyncio
from typing import List
from dotenv import load_dotenv
import logging
from google.adk.agents import Agent as ADKAgent, LlmAgent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.adk.tools import google_search
from google.adk.code_executors import BuiltInCodeExecutor
from google.genai import types

# Define variables required for Session setup and Agent execution
APP_NAME="calculator"
USER_ID="user1234"
SESSION_ID="session_code_exec_async"

# Agent Definition
code_agent = LlmAgent(
    name="calculator_agent",
    model="gemini-2.0-flash",
    code_executor=BuiltInCodeExecutor(),
    instruction="""You are a calculator agent.
    When given a mathematical expression, write and execute Python code
    to calculate the result.
    Return only the final numerical result as plain text, without
    markdown or code blocks.
    """,
    description="Executes Python code to perform calculations.",
)

# Agent Interaction (Async)
async def call_agent_async(query):

    # Session and Runner
    session_service = InMemorySessionService()
```

```

    session = await session_service.create_session(app_name=APP_NAME,
user_id=USER_ID, session_id=SESSION_ID)
    runner = Runner(agent=code_agent, app_name=APP_NAME,
session_service=session_service)

    content = types.Content(role='user',
parts=[types.Part(text=query)])
    print(f"\n--- Running Query: {query} ---")
    final_response_text = "No final text response captured."
    try:
        # Use run_async
        async for event in runner.run_async(user_id=USER_ID,
session_id=SESSION_ID, new_message=content):
            print(f"Event ID: {event.id}, Author: {event.author}")

            # --- Check for specific parts FIRST ---
            # has_specific_part = False
            if event.content and event.content.parts and
event.is_final_response():
                for part in event.content.parts: # Iterate through all
parts
                    if part.executable_code:
                        # Access the actual code string via .code
                        print(f"  Debug: Agent generated
code:\n``python\n{part.executable_code.code}\n``")
                        has_specific_part = True
                    elif part.code_execution_result:
                        # Access outcome and output correctly
                        print(f"  Debug: Code Execution Result:
{part.code_execution_result.outcome} -
Output:\n{part.code_execution_result.output}")
                        has_specific_part = True
                    # Also print any text parts found in any event for
debugging
                    elif part.text and not part.text.isspace():
                        print(f"  Text: '{part.text.strip()}'")
                        # Do not set has_specific_part=True here, as we
want the final response logic below

                        # --- Check for final response AFTER specific parts ---
text_parts = [part.text for part in event.content.parts
if part.text]
                        final_result = "".join(text_parts)
                        print(f"==> Final Agent Response: {final_result}")

    except Exception as e:
        print(f"ERROR during agent run: {e}")

```

```

print("-" * 30)

# Main async function to run the examples
async def main():
    await call_agent_async("Calculate the value of (5 + 7) * 3")
    await call_agent_async("What is 10 factorial?")

# Execute the main async function
try:
    nest_asyncio.apply()
    asyncio.run(main())
except RuntimeError as e:
    # Handle specific error when running asyncio.run in an already
    # running loop (like Jupyter/Colab)
    if "cannot be called from a running event loop" in str(e):
        print("\nRunning in an existing event loop (like
Colab/Jupyter).")
        print("Please run `await main()` in a notebook cell instead.")
        # If in an interactive environment like a notebook, you might
        # need to run:
        # await main()
    else:
        raise e # Re-raise other runtime errors

```

This script uses Google's Agent Development Kit (ADK) to create an agent that solves mathematical problems by writing and executing Python code. It defines an LlmAgent specifically instructed to act as a calculator, equipping it with the built_in_code_execution tool. The primary logic resides in the call_agent_async function, which sends a user's query to the agent's runner and processes the resulting events. Inside this function, an asynchronous loop iterates through events, printing the generated Python code and its execution result for debugging. The code carefully distinguishes between these intermediate steps and the final event containing the numerical answer. Finally, a main function runs the agent with two different mathematical expressions to demonstrate its ability to perform calculations.

Enterprise search: This code defines a Google ADK application using the google.adk library in Python. It specifically uses a VSearchAgent, which is designed to answer questions by searching a specified Vertex AI Search datastore. The code initializes a VSearchAgent named "q2_strategy_vsearch_agent", providing a description, the model to use ("gemini-2.0-flash-exp"), and the ID of the Vertex AI Search datastore. The DATASTORE_ID is expected to be set as an environment variable. It then sets up a Runner for the agent, using an InMemorySessionService to manage conversation

history. An asynchronous function `call_vsearch_agent_async` is defined to interact with the agent. This function takes a query, constructs a message content object, and calls the runner's `run_async` method to send the query to the agent. The function then streams the agent's response back to the console as it arrives. It also prints information about the final response, including any source attributions from the datastore. Error handling is included to catch exceptions during the agent's execution, providing informative messages about potential issues like an incorrect datastore ID or missing permissions. Another asynchronous function `run_vsearch_example` is provided to demonstrate how to call the agent with example queries. The main execution block checks if the `DATASTORE_ID` is set and then runs the example using `asyncio.run`. It includes a check to handle cases where the code is run in an environment that already has a running event loop, like a Jupyter notebook.

```
import asyncio
from google.genai import types
from google.adk import agents
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
import os

# --- Configuration ---
# Ensure you have set your GOOGLE_API_KEY and DATASTORE_ID
# environment variables
# For example:
# os.environ["GOOGLE_API_KEY"] = "YOUR_API_KEY"
# os.environ["DATASTORE_ID"] = "YOUR_DATASTORE_ID"

DATASTORE_ID = os.environ.get("DATASTORE_ID")

# --- Application Constants ---
APP_NAME = "vsearch_app"
USER_ID = "user_123" # Example User ID
SESSION_ID = "session_456" # Example Session ID

# --- Agent Definition (Updated with the newer model from the guide)
# ---
vsearch_agent = agents.VSearchAgent(
    name="q2_strategy_vsearch_agent",
    description="Answers questions about Q2 strategy documents using
Vertex AI Search.",
    model="gemini-2.0-flash-exp", # Updated model based on the guide's
examples
    datastore_id=DATASTORE_ID,
    model_parameters={"temperature": 0.0}
```

```

)

# --- Runner and Session Initialization ---
runner = Runner(
    agent=vsearch_agent,
    app_name=APP_NAME,
    session_service=InMemorySessionService(),
)

# --- Agent Invocation Logic ---
async def call_vsearch_agent_async(query: str):
    """Initializes a session and streams the agent's response."""
    print(f"User: {query}")
    print("Agent: ", end="", flush=True)

    try:
        # Construct the message content correctly
        content = types.Content(role='user',
            parts=[types.Part(text=query)])

        # Process events as they arrive from the asynchronous runner
        async for event in runner.run_async(
            user_id=USER_ID,
            session_id=SESSION_ID,
            new_message=content
        ):
            # For token-by-token streaming of the response text
            if hasattr(event, 'content_part_delta') and
event.content_part_delta:
                print(event.content_part_delta.text, end="",
flush=True)

            # Process the final response and its associated metadata
            if event.is_final_response():
                print() # Newline after the streaming response
                if event.grounding_metadata:
                    print(f" (Source Attributions:
{len(event.grounding_metadata.grounding_attributions)} sources
found)")
                else:
                    print(" (No grounding metadata found)")
                    print("-" * 30)

    except Exception as e:
        print(f"\nAn error occurred: {e}")
        print("Please ensure your datastore ID is correct and that the

```

```

service account has the necessary permissions.")
    print("-" * 30)

# --- Run Example ---
async def run_vsearch_example():
    # Replace with a question relevant to YOUR datastore content
    await call_vsearch_agent_async("Summarize the main points about
the Q2 strategy document.")
    await call_vsearch_agent_async("What safety procedures are
mentioned for lab X?")

# --- Execution ---
if __name__ == "__main__":
    if not DATASTORE_ID:
        print("Error: DATASTORE_ID environment variable is not set.")
    else:
        try:
            asyncio.run(run_vsearch_example())
        except RuntimeError as e:
            # This handles cases where asyncio.run is called in an
environment
            # that already has a running event loop (like a Jupyter
notebook).
            if "cannot be called from a running event loop" in str(e):
                print("Skipping execution in a running event loop.
Please run this script directly.")
            else:
                raise e

```

Overall, this code provides a basic framework for building a conversational AI application that leverages Vertex AI Search to answer questions based on information stored in a datastore. It demonstrates how to define an agent, set up a runner, and interact with the agent asynchronously while streaming the response. The focus is on retrieving and synthesizing information from a specific datastore to answer user queries.

Vertex Extensions: A Vertex AI extension is a structured API wrapper that enables a model to connect with external APIs for real-time data processing and action execution. Extensions offer enterprise-grade security, data privacy, and performance guarantees. They can be used for tasks like generating and running code, querying websites, and analyzing information from private datastores. Google provides prebuilt extensions for common use cases like Code Interpreter and Vertex AI Search, with the option to create custom ones. The primary benefit of extensions includes strong

enterprise controls and seamless integration with other Google products. The key difference between extensions and function calling lies in their execution: Vertex AI automatically executes extensions, whereas function calls require manual execution by the user or client.

At a Glance

What: LLMs are powerful text generators, but they are fundamentally disconnected from the outside world. Their knowledge is static, limited to the data they were trained on, and they lack the ability to perform actions or retrieve real-time information. This inherent limitation prevents them from completing tasks that require interaction with external APIs, databases, or services. Without a bridge to these external systems, their utility for solving real-world problems is severely constrained.

Why: The Tool Use pattern, often implemented via function calling, provides a standardized solution to this problem. It works by describing available external functions, or "tools," to the LLM in a way it can understand. Based on a user's request, the agentic LLM can then decide if a tool is needed and generate a structured data object (like a JSON) specifying which function to call and with what arguments. An orchestration layer executes this function call, retrieves the result, and feeds it back to the LLM. This allows the LLM to incorporate up-to-date, external information or the result of an action into its final response, effectively giving it the ability to act.

Rule of thumb: Use the Tool Use pattern whenever an agent needs to break out of the LLM's internal knowledge and interact with the outside world. This is essential for tasks requiring real-time data (e.g., checking weather, stock prices), accessing private or proprietary information (e.g., querying a company's database), performing precise calculations, executing code, or triggering actions in other systems (e.g., sending an email, controlling smart devices).

Visual summary:

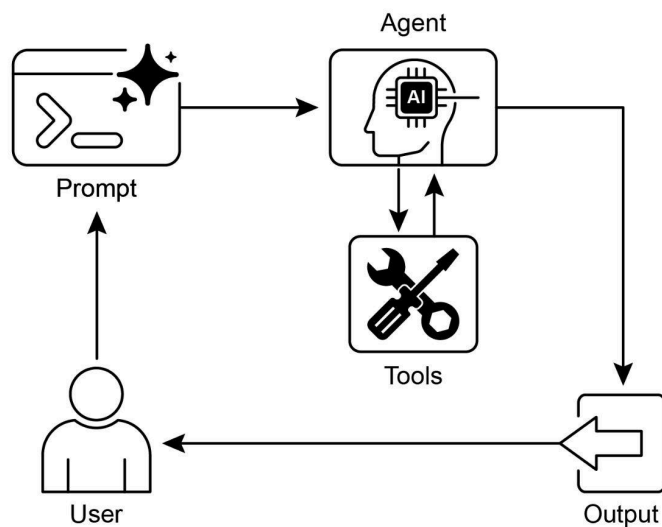


Fig.2: Tool use design pattern

Key Takeaways

- Tool Use (Function Calling) allows agents to interact with external systems and access dynamic information.
- It involves defining tools with clear descriptions and parameters that the LLM can understand.
- The LLM decides when to use a tool and generates structured function calls.
- Agentic frameworks execute the actual tool calls and return the results to the LLM.
- Tool Use is essential for building agents that can perform real-world actions and provide up-to-date information.
- LangChain simplifies tool definition using the `@tool` decorator and provides `create_tool_calling_agent` and `AgentExecutor` for building tool-using agents.

- Google ADK has a number of very useful pre-built tools such as Google Search, Code Execution and Vertex AI Search Tool.

Conclusion

The Tool Use pattern is a critical architectural principle for extending the functional scope of large language models beyond their intrinsic text generation capabilities. By equipping a model with the ability to interface with external software and data sources, this paradigm allows an agent to perform actions, execute computations, and retrieve information from other systems. This process involves the model generating a structured request to call an external tool when it determines that doing so is necessary to fulfill a user's query. Frameworks such as LangChain, Google ADK, and Crew AI offer structured abstractions and components that facilitate the integration of these external tools. These frameworks manage the process of exposing tool specifications to the model and parsing its subsequent tool-use requests. This simplifies the development of sophisticated agentic systems that can interact with and take action within external digital environments.

References

1. LangChain Documentation (Tools): <https://python.langchain.com/docs/integrations/tools/>
2. Google Agent Developer Kit (ADK) Documentation (Tools): <https://google.github.io/adk-docs/tools/>
3. OpenAI Function Calling Documentation: <https://platform.openai.com/docs/guides/function-calling>
4. CrewAI Documentation (Tools): <https://docs.crewai.com/concepts/tools>

Chapter 6: Planning

Intelligent behavior often involves more than just reacting to the immediate input. It requires foresight, breaking down complex tasks into smaller, manageable steps, and strategizing how to achieve a desired outcome. This is where the Planning pattern comes into play. At its core, planning is the ability for an agent or a system of agents to formulate a sequence of actions to move from an initial state towards a goal state.

Planning Pattern Overview

In the context of AI, it's helpful to think of a planning agent as a specialist to whom you delegate a complex goal. When you ask it to "organize a team offsite," you are defining the what—the objective and its constraints—but not the how. The agent's core task is to autonomously chart a course to that goal. It must first understand the initial state (e.g., budget, number of participants, desired dates) and the goal state (a successfully booked offsite), and then discover the optimal sequence of actions to connect them. The plan is not known in advance; it is created in response to the request.

A hallmark of this process is adaptability. An initial plan is merely a starting point, not a rigid script. The agent's real power is its ability to incorporate new information and steer the project around obstacles. For instance, if the preferred venue becomes unavailable or a chosen caterer is fully booked, a capable agent doesn't simply fail. It adapts. It registers the new constraint, re-evaluates its options, and formulates a new plan, perhaps by suggesting alternative venues or dates.

However, it is crucial to recognize the trade-off between flexibility and predictability. Dynamic planning is a specific tool, not a universal solution. When a problem's solution is already well-understood and repeatable, constraining the agent to a predetermined, fixed workflow is more effective. This approach limits the agent's autonomy to reduce uncertainty and the risk of unpredictable behavior, guaranteeing a reliable and consistent outcome. Therefore, the decision to use a planning agent versus a simple task-execution agent hinges on a single question: does the "how" need to be discovered, or is it already known?

Practical Applications & Use Cases

The Planning pattern is a core computational process in autonomous systems, enabling an agent to synthesize a sequence of actions to achieve a specified goal,